

- 1) Create a simple REST ful service in Spring Boot which returns the Response "Welcome to spring boot".

```
import org.springframework.web.bind.annotation.RestController;

@RestController
public class Controller {

    @GetMapping("/home")
    public String Default(){
        return "Welcome To Spring Boot";
    }
}
```

- 2) Create an Employee Bean(id, name, age) and service to perform different operations related to employee.
-

```

11
12     @Entity
13     @Setter
14     @Getter
15     @NoArgsConstructor
16     @AllArgsConstructor
17     @Table(name="employee")
18     public class EmployeeModel {
19         @Id
20         int id;
21         String name;
22         int age;
23     }
24

```

Created model class for employee

Created Service Layer With various Methods For Each type of request handling.....

```
@Service 2 usages
public class EmployeeService{

    @Autowired
    EmployeeRepo repo;

    public List<EmployeeModel> GetAllEmployee(){ no usages
        return repo.findAll();
    }

    public EmployeeModel getEmployeeById(Long id) { no usages
        return repo.findById(id).orElse( other: null);
    }

    public EmployeeModel saveEmployee(EmployeeModel employee) { no usages
        repo.save(employee);
        return repo.findById(employee.getId()).orElse( other: null);|
    }

    public void deleteEmployee(Long id) { no usages
        repo.deleteById(id);
    }

}
```

3 Implement GET http request for Employee to get list of employees.

```
public class Controller {

    @Autowired
    EmployeeService service;

    @GetMapping(Ⓢ"/home")
    public String Default(){
        return "Welcome To Spring Boot";
    }

    @GetMapping(Ⓢ"/getAllEmployee")
    public List<EmployeeModel> getAllEmp(){
        return service.getAllEmployee();
    }
}
```

4Implement GET http request using path variable top get one employee

5Implement POST http request for Employee to create a new employee.

```
@GetMapping(Ⓜ"/getAllEmployee")
public List<EmployeeModel> getAllEmp(){
    return service.GetAllEmployee();
}

@GetMapping(Ⓜ"/getById/{id}")
public EmployeeModel getById(@PathVariable long id){
    return service.getEmployeeById(id);
}

@PostMapping(Ⓜ"/addEmployee")
public EmployeeModel addEmp(@RequestParam EmployeeModel emp){
    return service.saveEmployee(emp);
}
```

Implement Exception Handling for resource not found

```
package com.ttn.restfulwebservices1.exception;

public class ResourceNotFoundException extends RuntimeException { no usages
    public ResourceNotFoundException(String message) { no usages
        super(message);
    }
}
```

Created class for resourceNotFoundException

```
package com.ttn.restfulwebservices1.exception;

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.ResponseStatus;
import org.springframework.web.bind.annotation.RestControllerAdvice;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public String handleResourceNotFound(ResourceNotFoundException ex){
        return ex.getMessage();
    }
}
```

Then created global exception handler with
@RestControllerAdvice

7) Implement DELETE http request for Employee to delete employee

```
}

@DeleteMapping("/{deltebyId}/{id}")
public ResponseEntity<String> deleteEmp(@PathVariable long id){
    service.deleteEmployee(id);
    return ResponseEntity.ok(body: "Employee Deleted Successfully");
}
}
```

8) Implement PUT http request for Employee to update employee

```
@PutMapping("/{editEmployee}/{id}")
public ResponseEntity<String> editEmployee(@PathVariable long id , @RequestBody EmployeeModel emp){
    EmployeeModel employeeSaved= service.updateEmployee(id, emp);
    return ResponseEntity.ok(body: "employee edited successfully");
}
}
```

9) Apply validation while create a new employee using POST http Request.

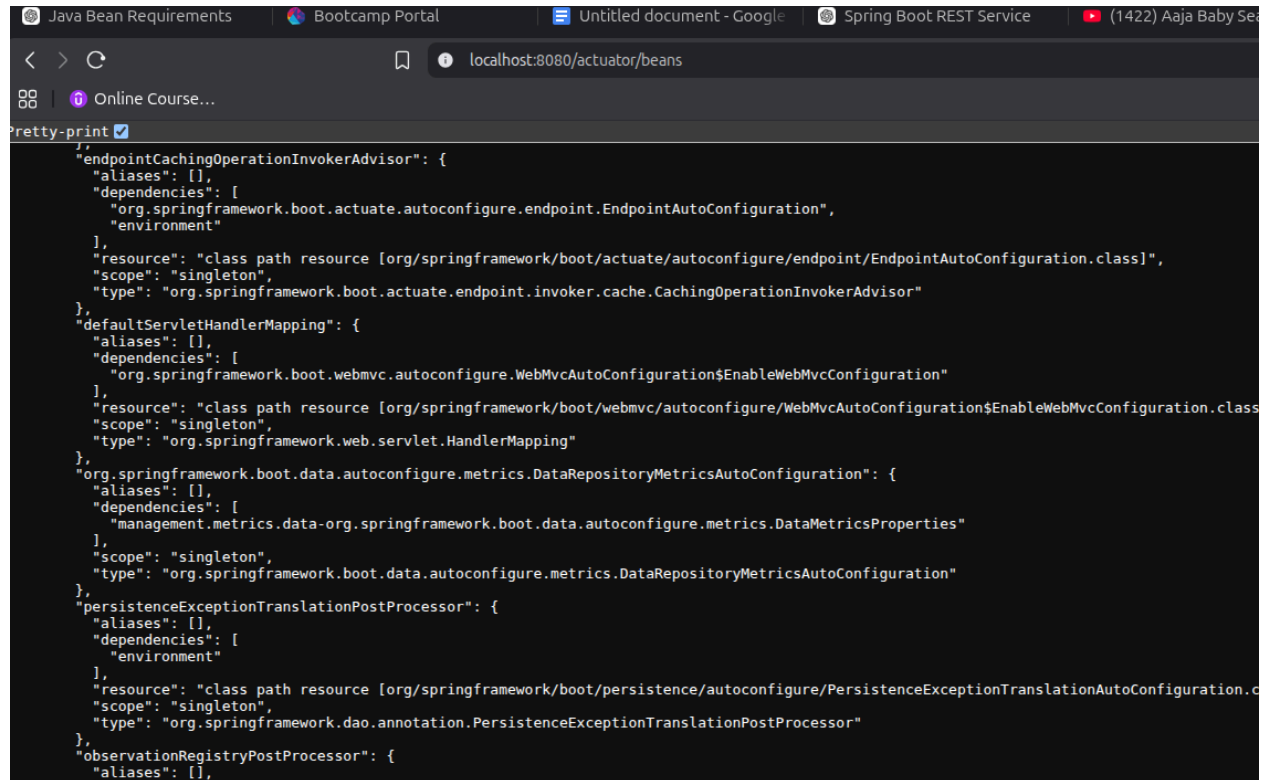
```
15 @Table(name="employee")
16 public class EmployeeModel {
17     @Id
18     @GeneratedValue(strategy = GenerationType.IDENTITY)
19     long id;
20     @NotBlank(message = "Name is mandatory")
21     @Size(min = 2, message = "Name must have at least 2 characters")
22     String name;
23
24     @NotNull(message = "Age is mandatory")
25     @Min(value = 18, message = "Age must be at least 18")
26     @Max(value = 60, message = "Age must not exceed 60")
27     int age;
28 }
29
```

In model class i added that validation and also updated in controller mapping like that

```
@PostMapping("/addEmployee")
Public ResponseEntity<EmployeeModel> addEmp(@Valid @RequestBody
EmployeeModel emp)
```

Here @Valid is Importat for validation

10) Configure actuator in your project to check the health of application and get the information about various beans configured in your application



```

{
  "endpointCachingOperationInvokerAdvisor": {
    "aliases": [],
    "dependencies": [
      "org.springframework.boot.actuate.autoconfigure.endpoint.EndpointAutoConfiguration",
      "environment"
    ],
    "resource": "class path resource [org/springframework/boot/actuate/autoconfigure/endpoint/EndpointAutoConfiguration.class]",
    "scope": "singleton",
    "type": "org.springframework.boot.actuate.endpoint.invoker.cache.CachingOperationInvokerAdvisor"
  },
  "defaultServletHandlerMapping": {
    "aliases": [],
    "dependencies": [
      "org.springframework.boot.webmvc.autoconfigure.WebMvcAutoConfiguration$EnableWebMvcConfiguration"
    ],
    "resource": "class path resource [org/springframework/boot/webmvc/autoconfigure/WebMvcAutoConfiguration$EnableWebMvcConfiguration.class]",
    "scope": "singleton",
    "type": "org.springframework.web.servlet.HandlerMapping"
  },
  "org.springframework.boot.data.autoconfigure.metrics.DataRepositoryMetricsAutoConfiguration": {
    "aliases": [],
    "dependencies": [
      "management.metrics.data-org.springframework.boot.data.autoconfigure.metrics.DataMetricsProperties"
    ],
    "scope": "singleton",
    "type": "org.springframework.boot.data.autoconfigure.metrics.DataRepositoryMetricsAutoConfiguration"
  },
  "persistenceExceptionTranslationPostProcessor": {
    "aliases": [],
    "dependencies": [
      "environment"
    ],
    "resource": "class path resource [org/springframework/boot/persistence/autoconfigure/PersistenceExceptionTranslationAutoConfiguration.class]",
    "scope": "singleton",
    "type": "org.springframework.dao.annotation.PersistenceExceptionTranslationPostProcessor"
  },
  "observationRegistryPostProcessor": {
    "aliases": [],
    "dependencies": [
      "environment"
    ],
    "resource": "class path resource [org/springframework/boot/actuate/autoconfigure/observation/observationRegistry/ObservationRegistryPostProcessor.class]",
    "scope": "singleton",
    "type": "org.springframework.boot.actuate.autoconfigure.observation.ObservationRegistryPostProcessor"
  }
}
```